Guía de Referencia Rápida - Examen Java

Índice

- Guía de Referencia Rápida - Examen Java
  - Índice
  - 📌 1. Validación de Identificadores
    - Reglas
    - Ejemplos (Validar identificadores)
  - 📌 2. Literales Numéricos
    - Sistemas de Numeración
    - Tipos de Datos y Sufijos
    - Cómo Identificar Tipo de Literal
    - Reglas de Validación
    - Ejemplos (Literales numéricos)
  - 📌 3. Literales de Tipo `char`
    - Reglas Clave
    - Secuencias de Escape Comunes
    - Ejemplos (Literales `Char`)
  - 📌 4. Conversiones de Tipos (Casting)
    - Jerarquía de Tipos
    - Cuándo se Necesita Casting
    - Tabla de Casting Requerido
    - Conversiones en Operaciones
    - Conversiones entre Primitivos y Objetos
    - Ejemplos de Casting
  - 📌 5. Operadores Lógicos y Relacionales
    - Operadores Lógicos
    - Operadores Relacionales
    - Incremento/Decremento
    - Ejemplos de Operadores
  - 📌 6. Expresiones Algorítmicas
    - Operaciones con Dígitos
    - Operaciones con Caracteres
    - Ejemplos Resueltos
  - 📌 7. Orden de Prioridad de Operadores
    - Ejemplos (Prioridad de Operadores)
  - 🎯 Checklist Final

📌 1. Validación de Identificadores

Reglas

- **Debe comenzar por:** letra (A-Z, a-z), guión bajo `_` o signo dólar `$`-
- No contener espacios, símbolos de puntuación ni carácteres especiales excepto `_` y `$`.

- No puede ser una clave reservada de java. Ej: `class`, `for`, `static`, `null`...

Ejemplos (Validar identificadores)

```
✅ int registro1;        // Válido
✅ int _contador;        // Válido
✅ int $precio;          // Válido
✅ int primerApellido;   // Válido (camelCase)

❌ int 1registro;        // No puede empezar con número
❌ int while;            // Palabra reservada
❌ int primer apellido;  // No puede tener espacios
❌ int primer-apellido;  // El guión no está permitido
```

📌 2. Literales Numéricos

Sistemas de Numeración

Sistema	Prefijo	Base	Ejemplo
Decimal	*(sin prefijo)*	10	`123`
Binario	`0b` o `0B`	2	`0b1010`
Octal	`0` *(cero inicial)*	8	`0123`
Hexadecimal	`0x` o `0X`	16	`0x1A3F`

Tipos de Datos y Sufijos

Tipo	Sufijo	Ejemplo
`int`	*(sin sufijo)*	`123`, `0x1A`
`long`	`L` o `l`	`123L`, `0b1010L`
`float`	`f` o `F`	`1.23f`, `1.23F`
`double`	*(sin sufijo)* o `d`/`D`	`1.23`, `1.23e5`

Cómo Identificar Tipo de Literal

Condición	Tipo
Lleva `.` o `e`/`E` (sin `F`)	`double`
Termina en `f` o `F`	`float`
Termina en `l` o `L`	`long`
Prefijo `0b`, `0`, `0x` (sin sufijo)	`int`

Condición	Tipo
Solo dígitos (sin sufijo ni prefijo)	`int`

Reglas de Validación

Regla	Válido	No Válido
No dos puntos decimales	`1.23`	`1.34.5`
Guiones bajos (_) solo entre dígitos	`1_234`	`_234`, `234_`, `1_.23`
Exponentes solo con decimales	`1.23e5`, `3e12`	`0x1Ae5`
No _ antes/después de `.` o `e`	`1_234.5e3`	`1234._5`, `3_e12`
Sistemas no decimales sin `.` ni `e`	`0x1A`, `0b101`	`0x1A.5`, `0b101e2`

Ejemplos (Literales numéricos)

```
☑ 0.5           // double, decimal
☑ 9.3e-12       // double, decimal (notación científica)
☑ 234567L       // long, decimal
☑ 0x87e3a       // int, hexadecimal
☑ 0b111         // int, binario
☑ 05_15         // int, octal
☑ 1234.2E3F     // float, decimal
☑ 1_2_3         // int, decimal

❌ 0.8E+0.8      // Exponente debe ser entero
❌ 12345678_L    // No puede haber _ antes del sufijo
❌ 018CDF        // Octal solo permite 0-7
❌ 0B1212        // Binario solo permite 0 y 1
❌ 3_e12         // No puede haber _ antes de 'e'
❌ 1.34.5        // Dos puntos decimales
```

📌 3. Literales de Tipo `char`

Reglas Clave

Regla	Válido	No Válido
Comillas simples `' '`	`'a'`	`"a"`, `a`
Solo **un** carácter	`'X'`	`'XY'`, `''`
Unicode: `\u` + 4 dígitos hex	`'\u0066'`	`'\u00G1'`
Secuencias de escape	`'\n'`, `'\t'`, `'\\'`, `'\''`	`'/n'`, `'\ñ'`
No comillas dobles	`'A'`	`"A"`

Secuencias de Escape Comunes

- `'\n'` → salto de línea
- `'\t'` → tabulación
- `'\\'` → barra invertida
- `'\''` → comilla simple
- `'\b'` → retroceso (backspace)

Ejemplos (Literales `Char`)

```
☑ 'a'           // Carácter simple
☑ '$'           // Símbolo
☑ '\n'          // Escape: salto de línea
☑ '\t'          // Escape: tabulación
☑ '\''          // Escape: comilla simple
☑ '\u0066'      // Unicode: 'f'
☑ '4'           // Carácter numérico
☑ 'ñ'           // Unicode válido

❌ "T"          // Comillas dobles → String
❌ '/n'         // No es escape válido
❌ '\ñ'         // No existe escape con ñ
❌ 'xyz'        // Más de un carácter
❌ k            // Falta comillas
❌ ''           // No puede estar vacío
```

📌 4. Conversiones de Tipos (Casting)

Jerarquía de Tipos

```
byte → short → int → long → float → double
        char ↗
```

Cuándo se Necesita Casting

Conversión	¿Casting?	Ejemplo
Tipo menor → mayor	NO (automático)	`int i = 5; long l = i;` ☑
Tipo mayor → menor	SÍ (explícito)	`double d = 3.14; int i = (int)d;`
`char` → `byte`/`short`	SÍ	`char c = 'a'; byte b = (byte)c;`
Operaciones entre enteros pequeños	Da `int`	`byte b1 = 1, b2 = 2; int r = b1 + b2;`

Tabla de Casting Requerido

De → A	¿Casting?
`int` → `short`, `byte`, `char`	☑ SÍ
`long` → `int`, `short`, `byte`, `char`	☑ SÍ
`float` → `int`, `long`, `short`, `byte`, `char`	☑ SÍ
`double` → `float`, `int`, `long`, `short`, `byte`, `char`	☑ SÍ
`char` → `byte`, `short`	☑ SÍ

Conversiones en Operaciones

Regla de promoción:

1. Si hay `double` → todo se convierte a `double`
2. Si hay `float` (sin `double`) → todo a `float`
3. Si hay `long` (sin `float`/`double`) → todo a `long`
4. Solo enteros pequeños (`byte`, `short`, `char`, `int`) → todo a `int`

Conversiones entre Primitivos y Objetos

DATO ORIGEN	DATO SALIDA	MÉTODO / EJEMPLO
Primitivo	**Primitivo**	**Casting** → `(tipo) variable` 🧩 *Ejemplo:* `int i = (int) 3.14;`
Objeto (String)	**Primitivo**	**parseTipo()** → `Tipo.parseTipo("valor")` 🧩 *Ejemplo:* `int n = Integer.parseInt("42");`
Primitivo	**Objeto (Wrapper)**	**valueOf()** → `Tipo.valueOf(valor)` 🧩 *Ejemplo:* `Integer num = Integer.valueOf(25);`
Primitivo	**Cadena (String)**	**String.valueOf()** → `String.valueOf(valor)` 🧩 *Ejemplo:* `String s = String.valueOf(123);`

Ejemplos de Casting

```
int i = 8;
float f = 2.5f;
double d = 10.0;
char c = 'A';
short s = 100;
byte b = 5;

// ☑ Conversiones automáticas (menor → mayor)
long l = i;              // int → long
double x = f;            // float → double

// ❌ Necesitan casting (mayor → menor)
int z = (int) d;         // double → int
short sh = (short) i;    // int → short
```

```
char ch = (char) (c + i);// int → char (operaciones dan int)

// Tipos resultantes en operaciones
i + c;           // int (char se convierte a int)
f - c;           // float (char → float)
d + i;           // double (int → double)
s + b;           // int (short + byte → int)
i * f * d;       // double (float * double → double)

// Asignaciones que requieren casting
char ch2 = c + i;        // ❌ int → char
char ch2 = (char)(c + i);// ☑ con casting

short y = s + 100000;    // ❌ int → short
short y = (short)(s + 100000); // ☑ con casting

float w = f + d;         // ❌ double → float
float w = (float)(f + d);// ☑ con casting

// Conversiones String ⇔ Primitivo
int edad = Integer.parseInt("25");           // String → int
double precio = Double.parseDouble("19.99"); // String → double
float valor = Float.parseFloat("1.23");      // String → float
String texto = String.valueOf(100);          // int → String

// Conversiones con Wrappers
Integer numObj = Integer.valueOf(42);        // int → Integer (Wrapper)
Double decimalObj = Double.valueOf(3.14);    // double → Double (Wrapper)
```

📌 5. Operadores Lógicos y Relacionales

Operadores Lógicos

Operador	Significado	Ejemplo
`&&`	AND (Y)	`a && b`
`||`	OR (O)	`a || b`
`!`	NOT (NO)	`!a`

⚡ Evaluación en cortocircuito:

- `&&` (AND): Si el primer operando es `false`, no evalúa el segundo (ya sabe que el resultado será `false`)
- `||` (OR): Si el primer operando es `true`, no evalúa el segundo (ya sabe que el resultado será `true`)

Diferencia entre `&&` vs `&` y `||` vs `|`:

- `&&` y `||` → Evaluación en cortocircuito (más eficiente)
- `&` y `|` → Siempre evalúan ambos operandos (menos eficiente)

Recomendación: Usa siempre `&&` y `||` para mejor rendimiento.

Operadores Relacionales

Operador	Significado
`==`	Igual a
`!=`	Diferente de
`<`	Menor que
`>`	Mayor que
`<=`	Menor o igual que
`>=`	Mayor o igual que

Incremento/Decremento

Operador	Descripción	Ejemplo
`++x`	Pre-incremento: incrementa **antes** de usar	`int x = 5; int y = ++x; // x=6, y=6`
`x++`	Post-incremento: incrementa **después** de usar	`int x = 5; int y = x++; // x=6, y=5`
`--x`	Pre-decremento: decrementa **antes** de usar	`int x = 5; int y = --x; // x=4, y=4`
`x--`	Post-decremento: decrementa **después** de usar	`int x = 5; int y = x--; // x=4, y=5`

Ejemplos de Operadores

```
boolean x = true;
boolean y = false;
boolean z = true;

// Operadores lógicos
(x && y) || z        // false || true → true
!x || (y && z)       // false || false → false
!(x || y) && z       // !true && true → false

int i = 8, j = 5;
float a = 0.005f, b = -0.01f;
char c = 'c', d = 'd';

// Operadores relacionales
i <= j               // false
c > d                // false ('c' ASCII=99 < 'd' ASCII=100)
a >= 0               // true
j != b               // true (tipos diferentes)
c == 99              // true (c en ASCII = 99)
```

```
// Incremento/decremento
++a > 0              // true (primero incrementa a=1.005, luego compara)
b-- < 1              // true (primero compara b=-0.01<1, luego decrementa)
```

📌 6. Expresiones Algorítmicas

Operaciones con Dígitos

Operación	Fórmula	Ejemplo
Última cifra de N	`N % 10`	`123 % 10 = 3`
Quitar última cifra	`N / 10`	`123 / 10 = 12`
Primera cifra (3 dígitos)	`N / 100`	`456 / 100 = 4`
Suma de dígitos (3 cifras)	`N/100 + (N%100)/10 + N%10`	`123 → 1+2+3 = 6`
Quitar primera cifra (5 dígitos)	`N % 10000`	`12345 % 10000 = 2345`

Operaciones con Caracteres

Operación	Código ASCII	Ejemplo
Letra mayúscula	`65-90` (`'A'` a `'Z'`)	`char c = 'B'; boolean esMayus = (c >= 65 && c <= 90);`
Letra minúscula	`97-122` (`'a'` a `'z'`)	`char c = 'b'; boolean esMinus = (c >= 97 && c <= 122);`
Dígito	`48-57` (`'0'` a `'9'`)	`char c = '5'; boolean esDigito = (c >= 48 && c <= 57);`

Ejemplos Resueltos

```
// 1. Precio entre 10€ y 50€ (inclusive 10, exclusive 50)
float precio = 25.5f;
boolean resultado = (precio >= 10 && precio < 50); // true

// 2. Comprobar si es letra mayúscula
char letra = 'M';
boolean esMayuscula = (letra >= 65 && letra <= 90); // true
// O también: (letra >= 'A' && letra <= 'Z')

// 3. Duplicar valor de X
int x = 10;
x = x * 2; // x ahora es 20

// 4. Diferencia entre (A*B) y (C+D)
int a = 10, b = 15, c = 20, d = 8;
```

```
int resultado = (a * b) - (c + d); // 150 - 28 = 122

// 5. Suma de últimas cifras de N y M
int N = 127, M = 345;
int suma = (N % 10) + (M % 10); // 7 + 5 = 12

// 6. Suma de dígitos de número de 3 cifras
int P = 456;
suma = P/100 + (P%100)/10 + P%10; // 4 + 5 + 6 = 15

// 7. Quitar primera cifra de número de 5 dígitos
int J = 12345;
J = J % 10000; // J = 2345
```

📌 7. Orden de Prioridad de Operadores

Cuando hay múltiples operadores en una expresión, se evalúan en este orden (de mayor a menor prioridad):

Prioridad	Operadores	Descripción
1	`expr++`, `expr--`	Post-incremento, post-decremento
2	`++expr`, `--expr`	Pre-incremento, pre-decremento
3	`*`, `/`, `%`	Multiplicación, división, módulo
4	`+`, `-`	Suma, resta
5	`<`, `>`, `<=`, `>=`	Operadores relacionales
6	`==`, `!=`	Igualdad, desigualdad
7	`&&`	AND lógico
8	`||`	OR lógico
9	`=`, `+=`, `-=`	Asignación

Ejemplos (Prioridad de Operadores)

```
// Ejemplo 1: Multiplicación antes que suma
int resultado = 5 + 3 * 2;  // resultado = 11 (no 16)
// Se evalúa: 5 + (3 * 2) = 5 + 6 = 11

// Ejemplo 2: Relacionales antes que lógicos
boolean b = 5 > 3 && 10 < 20;  // true
// Se evalúa: (5 > 3) && (10 < 20) = true && true = true

// Ejemplo 3: Pre-incremento vs post-incremento
int x = 5;
int y = ++x * 2;  // y = 12, x = 6
// Primero incrementa x a 6, luego multiplica: 6 * 2 = 12
```

```
int a = 5;
int b = a++ * 2;  // b = 10, a = 6
// Primero multiplica: 5 * 2 = 10, luego incrementa a a 6

// Ejemplo 4: Paréntesis cambian el orden
int resultado2 = (5 + 3) * 2;  // resultado2 = 16
// Se evalúa primero lo de los paréntesis: (8) * 2 = 16

// Ejemplo 5: Operadores de asignación compuestos
int n = 10;
n += 5 * 2;  // n = 20
// Se evalúa: n = n + (5 * 2) = 10 + 10 = 20
```

Consejo: Cuando tengas dudas, usa **paréntesis** para dejar claro el orden de evaluación.

Checklist Final

Antes del examen, asegúrate de saber:

- [ ] Reglas de identificadores válidos
- [ ] Prefijos de sistemas de numeración (0b, 0, 0x)
- [ ] Sufijos de tipos (L, F, d)
- [ ] Cuándo usar casting (mayor → menor)
- [ ] Jerarquía de tipos en operaciones
- [ ] Secuencias de escape de char
- [ ] Operadores lógicos (&&, ||, !) y diferencia con & y |
- [ ] Pre vs post incremento/decremento
- [ ] Operador módulo (%) para extraer cifras
- [ ] División entera (/) para quitar cifras
- [ ] Orden de prioridad de operadores
- [ ] Conversiones String ↔ Primitivo (parseInt, valueOf)